WEEK 3 · LESSON 16 OF 24 · HANDOUT

Transfer Learning: Standing on Big Models

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=nB15sW-mkHA — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191 (2024): Convolutional Neural Networks — https://www.youtube.com/watch?v=2xqkSUhmmXU

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

Your goal in this lesson: Explain transfer learning, show its benefit when data is scarce, and adapt a pretrained ResNet from torchvision to a new task by freezing or fine-tuning.

Don't start from zero

Training a strong image model from scratch needs hundreds of thousands of labelled photos and many GPU hours. But the early layers of any good image model learn **general** patterns — edges, corners, textures, shapes — that are useful for almost every visual task.

Transfer learning takes a model already trained on a huge dataset (usually **ImageNet**: 1.2 million photos in 1,000 classes) and re-uses it for your task. You keep its "eyes" (the feature layers) and replace only its "brain" (the final classifier). This is how most real projects are built — including crop-disease apps trained on a few thousand leaf photos.

Two ways to transfer

Method	What trains	When
Feature extraction (freeze)	Only the new head	Very little data; fast; works on a CPU
Fine-tuning	The head, then some or all layers with a small learning rate	More data, or a task very different from ImageNet

See the benefit with our own "pretrained" model

We first train a CNN on circles, squares and triangles. Then we face a **new** task — rings, diamonds and crosses — with only **30** labelled images.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

def make_shapes(n, kinds=("circle", "square", "triangle"), size=32, seed=0):
    # n noisy grey images of random shapes at random places and sizes
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    images = np.zeros((n, size, size), dtype=np.float32)
    labels = rng.integers(0, len(kinds), n)
    for i, lab in enumerate(labels):
        r = rng.uniform(5, 10)
        cx, cy = rng.uniform(r + 1, size - r - 1, 2)
        dx, dy = xx - cx, yy - cy
        kind = kinds[lab]
        if kind == "circle":
            mask = dx ** 2 + dy ** 2 <= r ** 2
        elif kind == "square":
            mask = (abs(dx) <= r * 0.85) & (abs(dy) <= r * 0.85)
        elif kind == "triangle":
            mask = (dy >= -r) & (dy <= r) & (abs(dx) <= (dy + r) / 2)
        elif kind == "ring":
            mask = (dx ** 2 + dy ** 2 <= r ** 2) & (dx ** 2 + dy ** 2 >= (0.55 * r) ** 2)
        elif kind == "diamond":
            mask = abs(dx) + abs(dy) <= r
        else:  # cross
            mask = ((abs(dx) <= 0.3 * r) & (abs(dy) <= r)) | ((abs(dy) <= 0.3 * r) & (abs(dx)
<= r))
        images[i][mask] = rng.uniform(0.6, 1.0)
        images[i] += rng.normal(0, 0.15, (size, size))
    images = torch.tensor(np.clip(images, 0, 1)).unsqueeze(1)   # (n, 1 channel, 32, 32)
    return images, torch.tensor(labels)

X_all, y_all = make_shapes(3000)
X_train, y_train = X_all[:2400], y_all[:2400]
X_test, y_test = X_all[2400:], y_all[2400:]
names = ["circle", "square", "triangle"]

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, lab in zip(axes, X_train, y_train):
    ax.imshow(img[0], cmap="gray")
    ax.set_title(names[lab])
    ax.axis("off")
plt.show()
print(X_train.shape, torch.bincount(y_train))
```

```
class ShapeCNN(nn.Module):
    def __init__(self, n_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 32 ->
16
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 16 -> 8
            nn.Flatten())
        self.head = nn.Sequential(nn.Linear(32 * 8 * 8, 64), nn.ReLU(), nn.Linear(64,
n_classes))

    def forward(self, x):
        return self.head(self.features(x))

def train_model(model, X, y, epochs=5, batch=64, lr=1e-3, params=None, augment=None):
    opt = torch.optim.Adam(params if params is not None else model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X))
        for start in range(0, len(perm), batch):
            idx = perm[start:start + batch]
            xb = X[idx]
            if augment is not None:
                xb = torch.stack([augment(img) for img in xb])
            opt.zero_grad()
            F.cross_entropy(model(xb), y[idx]).backward()
            opt.step()
    return model

def accuracy(model, X, y):
    model.eval()
    with torch.no_grad():
        return round((model(X).argmax(dim=1) == y).float().mean().item(), 3)
```

```
import copy

# 1. "Pretrain" on the big, old task
torch.manual_seed(0)
pretrained = train_model(ShapeCNN(), X_train, y_train, epochs=5)
print("old task accuracy:", accuracy(pretrained, X_test, y_test))

# 2. The new task: only 30 training images
new_kinds = ("ring", "diamond", "cross")
X_new, y_new = make_shapes(30, kinds=new_kinds, seed=1)
X_new_test, y_new_test = make_shapes(600, kinds=new_kinds, seed=2)

def new_head():
    return nn.Sequential(nn.Linear(32 * 8 * 8, 64), nn.ReLU(), nn.Linear(64, 3))

# (a) from scratch
torch.manual_seed(0)
scratch = train_model(ShapeCNN(), X_new, y_new, epochs=60, batch=32)

# (b) transfer: keep the learned features frozen, train a new head
frozen = copy.deepcopy(pretrained)
frozen.head = new_head()
for p in frozen.features.parameters():
    p.requires_grad = False
torch.manual_seed(0)
frozen = train_model(frozen, X_new, y_new, epochs=60, batch=32,
params=frozen.head.parameters())

print("from scratch:      ", accuracy(scratch, X_new_test, y_new_test))
print("transfer (frozen): ", accuracy(frozen, X_new_test, y_new_test))
```

With only 30 images, the network trained from scratch gets about **0.43** (only a little better than guessing one of three). The transferred network gets about **0.56** — the edge and corner detectors learned on the old shapes carry over to the new ones. With 90 images the gap stays large (about 0.66 vs 0.79 in our tests).

A real pretrained network: ResNet-18

torchvision includes dozens of networks pretrained on ImageNet. In Colab, the first line below downloads the weights (about 45 MB) automatically:

```
from torchvision import models

weights = models.ResNet18_Weights.DEFAULT
resnet = models.resnet18(weights=weights)
preprocess = weights.transforms()          # the resize/normalise these weights expect
print(resnet.fc)                           # the final layer: 512 features -> 1000 ImageNet
classes
print("parameters:", sum(p.numel() for p in resnet.parameters()))
```

ResNet-18 has about **11.7 million** parameters. To adapt it to, say, 4 cassava conditions:

```
for p in resnet.parameters():              # freeze everything ...
    p.requires_grad = False
resnet.fc = nn.Linear(resnet.fc.in_features, 4)   # ... then add a new, trainable head

trainable = sum(p.numel() for p in resnet.parameters() if p.requires_grad)
print("trainable parameters:", trainable)

fake_photos = torch.rand(2, 3, 224, 224)  # two colour images
print(resnet(preprocess(fake_photos)).shape)
```

Only **2,052** of the 11.7 million parameters now train (512 × 4 weights + 4 biases), so even a CPU can learn the new head in minutes. `preprocess` resizes and normalises images exactly the way the network saw them during pretraining — always use it.

Fine-tuning, step by step

1. Train the new head with everything else frozen (a few epochs).
2. Unfreeze the last block (`resnet.layer4`) — or everything.
3. Continue with a **small** learning rate (for example 1e-4), so the pretrained knowledge is adjusted, not destroyed.

```
for p in resnet.layer4.parameters():
    p.requires_grad = True
opt = torch.optim.Adam([
    {"params": resnet.layer4.parameters(), "lr": 1e-4},
    {"params": resnet.fc.parameters(), "lr": 1e-3},
])
print("trainable now:", sum(p.numel() for p in resnet.parameters() if p.requires_grad))
```

Where pretrained models come from

- **torchvision.models** — ResNet, EfficientNet, MobileNet (small, for phones), ConvNeXt, vision transformers.
- **Hugging Face Hub** and **timm** — thousands more, for images, text and audio.
- Always check the **licence** and what data the model was trained on — a model trained only on European faces or North American crops may not work well on Rwandan data.

Key terms

- **Transfer learning** — reusing a model trained on one task for another.
- **Pretrained model** — a network already trained on a large dataset such as ImageNet.
- **Freezing** — stopping some layers from updating (`requires_grad = False`).
- **Fine-tuning** — continuing to train pretrained layers with a small learning rate.
- **Feature extractor** — the frozen pretrained layers that turn images into features.

Remember this: With little data, borrow a pretrained network's eyes and train only a new head; fine-tune gently when you have more data.

Name: Date:

Activity

Task 1

Fine-tune all layers of the transferred shape model (no freezing). Is it better or worse than the frozen version with 30 images?

Task 2

Repeat the experiment with 90 and 300 new images. When does training from scratch catch up?

Task 3

Load `models.mobilenet_v3_small` and count its parameters. Why might it suit a phone app?

Task 4

Write down a plan to adapt ResNet-18 to 5 types of local fruit using 100 photos of each.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is transfer learning?
   A. Moving a model to a GPU
   B. Copying data between computers
   C. Reusing a model trained on one task for a new task
   D. Training without data
2. Why do pretrained features transfer to new image tasks?
   A. The labels are copied
   B. All images are the same
   C. They don't
   D. Early layers learn general patterns such as edges and textures
3. With very little data, what is usually the best approach?
   A. Freeze the pretrained layers and train only a new head
   B. Use no validation
   C. Train a huge network from scratch
   D. Unfreeze everything with a big learning rate
4. How do you freeze a layer in PyTorch?
   A. Delete it
   B. Call model.eval()
   C. Set requires_grad = False on its parameters
   D. Lower the batch size
5. Why use a small learning rate when fine-tuning?
   A. Because GPUs require it
   B. To freeze the head
   C. To train faster
   D. To adjust the pretrained knowledge without destroying it
6. What does weights.transforms() give you?
   A. The labels
   B. A new dataset
   C. The model's licence
   D. The exact resizing and normalisation the pretrained network expects

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.